



Faculty of Technology and Engineering

Computer Science and Engineering

Practical

Academic Year	:	2025-26	Semester	:	6
Course code	:	CSE312	Course name	:	Design of language processing

Practical - 7

1. Objective:

Develop a YACC program to validate input strings based on the given grammar. The program should parse the string using the grammar rules and determine whether the string is valid or invalid.

$$S \rightarrow i E t S S' \mid a$$

$$S' \rightarrow e S \mid \varepsilon$$

$$E \rightarrow b$$

2. Program Code:

```
#include <iostream>
#include <map>
#include <set>
#include <vector>
#include <stack>
#include <string>
#include <iomanip>
#include <sstream>
#include <algorithm>
```

```
using namespace std;
```

```
// — Types
```

```
using Symbol = string;
using Production = vector<Symbol>;
```

```
const Symbol EPSILON = "ε";
const Symbol END_OF_INPUT = "$";
```

```
struct Grammar {
    vector<Symbol> non_terminals;
    vector<Symbol> terminals;
    Symbol start;
    map<Symbol, vector<Production>> rules; // NT -> list of productions
```

```
};
```

```
// — Grammar definition
```

```
Grammar buildGrammar() {
    Grammar g;
    g.start = "S";
    g.non_terminals = {"S", "S'", "T"};
    g.terminals = {"(", ")", "a", "b", "c", "$"};

    // S -> T S'
    g.rules["S"].push_back({"T", "S'"});

    // S' -> T S' | ε
    g.rules["S'"].push_back({"T", "S'"});
    g.rules["S'"].push_back({EPSILON});

    // T -> ( S ) | a | b | c
    g.rules["T"].push_back({"(", "S", "}")});
    g.rules["T"].push_back({"a"});
    g.rules["T"].push_back({"b"});
    g.rules["T"].push_back({"c"});

    return g;
}
```

```
// — FIRST / FOLLOW
```

```
bool isTerminal(const Grammar& g, const Symbol& s) {
    return find(g.non_terminals.begin(), g.non_terminals.end(), s)
        == g.non_terminals.end();
}

map<Symbol, set<Symbol>> computeFirst(const Grammar& g) {
    map<Symbol, set<Symbol>> first;

    // Terminals: FIRST(a) = {a}
    for (const auto& t : g.terminals) first[t].insert(t);
    first[EPSILON].insert(EPSILON);

    bool changed = true;
    while (changed) {
        changed = false;
        for (const auto& nt : g.non_terminals) {
            for (const auto& prod : g.rules.at(nt)) {
                // Add FIRST of each symbol until one has no ε
                bool allHaveEps = true;
                for (const auto& sym : prod) {
                    size_t before = first[nt].size();
                    for (const auto& f : first[sym]) {
                        if (f != EPSILON) first[nt].insert(f);
                    }
                }
                if (first[nt].size() > before) changed = true;
            }
        }
    }
}
```

```

    }
    if (first[nt].size() != before) changed = true;

    if (first[sym].find(EPSILON) == first[sym].end()) {
        allHaveEps = false;
        break;
    }
}
if (allHaveEps) {
    if (first[nt].insert(EPSILON).second) changed = true;
}
}
}
return first;
}

// FIRST of a sequence of symbols (used for table construction)
set<Symbol> firstOfSequence(const vector<Symbol>& seq,
                           const map<Symbol, set<Symbol>>& first) {
    set<Symbol> result;
    bool allHaveEps = true;
    for (const auto& sym : seq) {
        for (const auto& f : first.at(sym)) {
            if (f != EPSILON) result.insert(f);
        }
        if (first.at(sym).find(EPSILON) == first.at(sym).end()) {
            allHaveEps = false;
            break;
        }
    }
    if (allHaveEps) result.insert(EPSILON);
    return result;
}

map<Symbol, set<Symbol>> computeFollow(const Grammar& g,
                                       const map<Symbol, set<Symbol>>& first) {
    map<Symbol, set<Symbol>> follow;
    follow[g.start].insert(END_OF_INPUT);

    bool changed = true;
    while (changed) {
        changed = false;
        for (const auto& nt : g.non_terminals) {
            for (const auto& prod : g.rules.at(nt)) {
                for (size_t i = 0; i < prod.size(); ++i) {
                    const Symbol& B = prod[i];
                    if (isTerminal(g, B) || B == EPSILON) continue;

                    //  $\beta = \text{prod}[i+1 \dots \text{end}]$ 
                    vector<Symbol> beta(prod.begin() + i + 1, prod.end());

```

```

    set<Symbol> firstBeta;
    if (beta.empty()) {
        firstBeta.insert(EPSILON);
    } else {
        firstBeta = firstOfSequence(beta, first);
    }

    // Add FIRST( $\beta$ ) \ { $\epsilon$ } to FOLLOW(B)
    for (const auto& f : firstBeta) {
        if (f != EPSILON) {
            if (follow[B].insert(f).second) changed = true;
        }
    }
    // If  $\epsilon \in \text{FIRST}(\beta)$ , add FOLLOW(nt)
    if (firstBeta.count(EPSILON)) {
        for (const auto& f : follow[nt]) {
            if (follow[B].insert(f).second) changed = true;
        }
    }
}
}
}
}
return follow;
}

// — Parsing Table

```

```

// table[NT][terminal] = production index (-1 = error, -2 = conflict)
using ParseTable = map<Symbol, map<Symbol, int>>;

ParseTable buildParseTable(const Grammar& g,
    const map<Symbol, set<Symbol>>& first,
    const map<Symbol, set<Symbol>>& follow,
    bool& isLL1) {
    ParseTable table;
    isLL1 = true;

    // Initialise to -1 (error)
    for (const auto& nt : g.non_terminals)
        for (const auto& t : g.terminals)
            table[nt][t] = -1;

    for (const auto& nt : g.non_terminals) {
        const auto& prods = g.rules.at(nt);
        for (int idx = 0; idx < (int)prods.size(); ++idx) {
            const auto& prod = prods[idx];

            set<Symbol> firstAlpha = firstOfSequence(prod, first);

```

```

// For each terminal a in FIRST( $\alpha$ )
for (const auto& a : firstAlpha) {
    if (a == EPSILON) continue;
    if (table[nt][a] != -1) { isLL1 = false; table[nt][a] = -2; }
    else
        table[nt][a] = idx;
}

// If  $\epsilon \in \text{FIRST}(\alpha)$ , for each b in FOLLOW(NT)
if (firstAlpha.count(EPSILON)) {
    for (const auto& b : follow.at(nt)) {
        if (table[nt][b] != -1) { isLL1 = false; table[nt][b] = -2; }
        else
            table[nt][b] = idx;
    }
}
}
}
return table;
}

// — Display helpers

string productionToString(const Symbol& nt, const Production& prod) {
    string s = nt + " -> ";
    for (const auto& sym : prod) s += sym + " ";
    if (!s.empty() && s.back() == ' ') s.pop_back();
    return s;
}

void printFirstFollow(const Grammar& g,
                    const map<Symbol, set<Symbol>>& first,
                    const map<Symbol, set<Symbol>>& follow) {
    cout << "\n===== FIRST & FOLLOW Sets =====\n";
    cout << left << setw(8) << "NT"
        << setw(35) << "FIRST"
        << "FOLLOW\n";
    cout << string(70, '-') << "\n";
    for (const auto& nt : g.non_terminals) {
        string f1, f2;
        for (const auto& s : first.at(nt)) f1 += s + " ";
        for (const auto& s : follow.at(nt)) f2 += s + " ";
        cout << left << setw(8) << nt
            << setw(35) << ("{" + f1 + "}")
            << ("{" + f2 + "}")
            << "\n";
    }
}

void printParseTable(const Grammar& g, const ParseTable& table) {
    // Terminals without $
    vector<Symbol> cols;
    for (const auto& t : g.terminals)

```

```

    if (t != END_OF_INPUT) cols.push_back(t);
    cols.push_back(END_OF_INPUT);

    int colW = 18;
    cout << "\n===== Predictive Parsing Table =====\n";
    cout << left << setw(6) << "NT";
    for (const auto& t : cols) cout << setw(colW) << t;
    cout << "\n" << string(6 + colW * cols.size(), '-') << "\n";

    for (const auto& nt : g.non_terminals) {
        cout << left << setw(6) << nt;
        for (const auto& t : cols) {
            int idx = table.at(nt).at(t);
            string cell;
            if (idx == -1) cell = "—";
            else if (idx == -2) cell = "CONFLICT";
            else cell = productionToString(nt, g.rules.at(nt)[idx]);
            cout << setw(colW) << cell;
        }
        cout << "\n";
    }
}

// — LL(1) Parsing

```

```

bool parseString(const string& input, const Grammar& g,
    const ParseTable& table, bool verbose = true) {
    // Tokenise: each character is a token; append $
    vector<Symbol> tokens;
    for (char ch : input) {
        string tok(1, ch);
        tokens.push_back(tok);
    }
    tokens.push_back(END_OF_INPUT);

    stack<Symbol> stk;
    stk.push(END_OF_INPUT);
    stk.push(g.start);

    int pos = 0;

    if (verbose) {
        cout << "\n--- Parsing: \"" << input << "\" ---\n";
        cout << left << setw(30) << "Stack" << setw(20) << "Input" << "Action\n";
        cout << string(70, '-') << "\n";
    }

    while (!stk.empty()) {
        Symbol top = stk.top();

        // Build display strings

```

```

if (verbose) {
    string stackStr, inputStr;
    stack<Symbol> tmp = stk;
    vector<Symbol> stackVec;
    while (!tmp.empty()) { stackVec.push_back(tmp.top()); tmp.pop(); }
    for (int i = (int)stackVec.size()-1; i >= 0; --i) stackStr += stackVec[i];
    for (int i = pos; i < (int)tokens.size(); ++i) inputStr += tokens[i];
    cout << left << setw(30) << stackStr << setw(20) << inputStr;
}

if (top == END_OF_INPUT) {
    if (tokens[pos] == END_OF_INPUT) {
        if (verbose) cout << "Accept\n";
        return true;
    } else {
        if (verbose) cout << "Reject (extra input)\n";
        return false;
    }
}

// Top is a terminal
if (isTerminal(g, top)) {
    if (top == tokens[pos]) {
        if (verbose) cout << "Match '" << top << "'\n";
        stk.pop();
        ++pos;
    } else {
        if (verbose) cout << "Error: expected '" << top
            << "' got '" << tokens[pos] << "'\n";
        return false;
    }
    continue;
}

// Top is a non-terminal
const Symbol& cur = tokens[pos];
int idx = (table.count(top) && table.at(top).count(cur))
    ? table.at(top).at(cur) : -1;

if (idx < 0) {
    if (verbose) cout << "Error: no rule for [" << top << ", " << cur << "]\n";
    return false;
}

const Production& prod = g.rules.at(top)[idx];
if (verbose) cout << productionToString(top, prod) << "\n";

stk.pop();
if (!(prod.size() == 1 && prod[0] == EPSILON)) {
    for (int i = (int)prod.size()-1; i >= 0; --i)
        stk.push(prod[i]);
}

```

```

    }
}
return false;
}

// — Main

—
int main() {
    Grammar g = buildGrammar();

    // Print grammar
    cout << "==== Grammar =====\n";
    for (const auto& nt : g.non_terminals) {
        for (size_t i = 0; i < g.rules.at(nt).size(); ++i) {
            cout << " " << productionToString(nt, g.rules.at(nt)[i]) << "\n";
        }
    }

    auto first = computeFirst(g);
    auto follow = computeFollow(g, first);

    printFirstFollow(g, first, follow);

    bool isLL1 = false;
    ParseTable table = buildParseTable(g, first, follow, isLL1);

    printParseTable(g, table);

    cout << "\n>>> Grammar is " << (isLL1 ? "LL(1)" : "NOT LL(1)") << " <<<\n";

    if (!isLL1) {
        cout << "Cannot validate strings: grammar is not LL(1).\n";
        return 0;
    }

    // — Test cases

    vector<string> testCases = {
        "abc", "ac", "(abc)", "c", "(ac)",
        "a", "()", "(ab)", "abcabc", "b"
    };

    cout << "\n==== String Validation =====\n";
    for (const auto& s : testCases) {
        bool valid = parseString(s, g, table, false);
        cout << " \" << left << setw(10) << (s + "\"")
            << " -> " << (valid ? "Valid string" : "Invalid string") << "\n";
    }

    // Interactive mode

```



```

cout << "\n===== Interactive Mode =====\n";
cout << "Enter a string to validate (or 'quit' to exit):\n";
string line;
while (true) {
    cout << "> ";
    if (!getline(cin, line) || line == "quit" || line == "q") break;
    bool valid = parseString(line, g, table, true);
    cout << "\nResult: " << (valid ? "Valid string" : "Invalid string") << "\n\n";
}

return 0;
}

```

3.Output:

```
debdoottmanna@Debdoots-MacBook-Air DLP % g++ -std=c++17 -o 8 8.cpp
```

```
debdoottmanna@Debdoots-MacBook-Air DLP % ./8
```

```
===== Grammar =====
```

```

S -> T S'
S' -> T S'
S' -> ε
T -> ( S )
T -> a
T -> b
T -> c

```

```
===== FIRST & FOLLOW Sets =====
```

NT	FIRST	FOLLOW
S	{ (a b c }	{ \$) }
S'	{ (a b c ε }	{ \$) }
T	{ (a b c }	{ \$ () a b c }

```
===== Predictive Parsing Table =====
```

NT	(	)	a	b	c	\$
S	S -> T S'	-	S -> T S'	S -> T S'	S -> T S'	-
S'	S' -> T S'	S' -> ε	S' -> T S'	S' -> T S'	S' -> T S'	S' -> ε
T	T -> (S)	-	T -> a	T -> b	T -> c	-

```
>>> Grammar is LL(1) <<<
```

```
===== String Validation =====
```

```

"abc"    -> Valid string
"ac"     -> Valid string
"(abc)"  -> Valid string
"c"      -> Valid string
"(ac)"   -> Valid string
"a"      -> Valid string
"()"     -> Invalid string
"(ab)"   -> Valid string
"abcabc" -> Valid string
"b"      -> Valid string

```

```
===== Interactive Mode =====
```

```
Enter a string to validate (or 'quit' to exit):
```

```
> (ab)
```

```
--- Parsing: "(ab)" ---
```

Stack	Input	Action
\$S	(ab)\$	S -> T S'
\$S'T	(ab)\$	T -> (S)
\$S')S(	(ab)\$	Match '('
\$S')S	ab)\$	S -> T S'

\$S')S'T	ab)\$	T → a
\$S')S'a	ab)\$	Match 'a'
\$S')S'	b)\$	S' → T S'
\$S')S'T	b)\$	T → b
\$S')S'b	b)\$	Match 'b'
\$S')S'	)\$	S' → ε
\$S')	)\$	Match ')'
\$S'	\$	S' → ε
\$	\$	Accept

Result: Valid string

> ()

--- Parsing: "()" ---

Stack	Input	Action
\$S	()\$	S → T S'
\$S'T	()\$	T → (S)
\$S')S(	()\$	Match '('
\$S')S	)\$	Error: no rule for [S,)]

Result: Invalid string

> abcababc

--- Parsing: "abcababc" ---

Stack	Input	Action
\$S	abcababc\$	S → T S'
\$S'T	abcababc\$	T → a
\$S'a	abcababc\$	Match 'a'
\$S'	bcababc\$	S' → T S'
\$S'T	bcababc\$	T → b
\$S'b	bcababc\$	Match 'b'
\$S'	cababc\$	S' → T S'
\$S'T	cababc\$	T → c
\$S'c	cababc\$	Match 'c'
\$S'	abc\$	S' → T S'
\$S'T	abc\$	T → a
\$S'a	abc\$	Match 'a'
\$S'	bc\$	S' → T S'
\$S'T	bc\$	T → b
\$S'b	bc\$	Match 'b'
\$S'	c\$	S' → T S'
\$S'T	c\$	T → c
\$S'c	c\$	Match 'c'
\$S'	\$	S' → ε
\$	\$	Accept

Result: Valid string

> quit